

Lesson 1: Intro to Microcontrollers

Overview

- Embedded systems in everyday life
- Microcontroller vs computer
- GPIO basics
- Hands-on activity
 - Building and LED traffic light
 - Troubleshooting with a multimeter

Embedded Systems

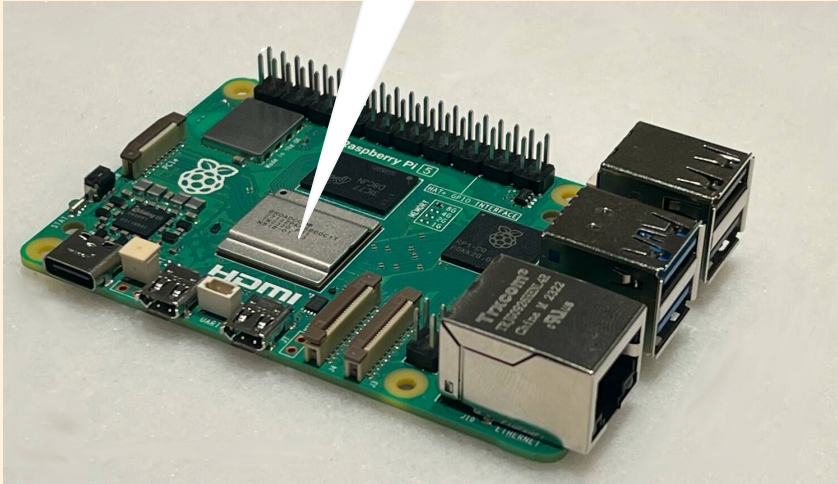
An *embedded system* is a specialized computer system designed to perform a dedicated function, oftentimes with high-speed or low-power requirements.

Examples in everyday life:

- Pacemaker
- Thermostats
- Industrial robots
- Satellites
- Barcode scanners
- Automatic doors
- ATMs
- Elevators
- Solar arrays
- Microwave ovens

Computer vs Microcontroller

Microprocessor



Raspberry pi (a.k.a “single-board computer”)

- An integrated system with a microprocessor, an operating system, external memory, and interfaces for peripherals

Microcontroller



Arduino

- A platform that allows easy interfacing with the all-in-one microcontroller (ATmega 328P)

Computer vs Microcontroller

- Higher processing speed
 - > 1 Ghz
- More storage
 - SD cards, USB drives
- More memory (RAM)
 - > 1 Gb

- More power consumption
 - > 500 mA draw
 - Difficult to power with batteries
- Boot time
 - > 10 seconds

- Other
 - Internet access
 - WiFi
 - Can program directly

- Slower processing speed
 - < 100 Mhz
- Less storage
 - < 1 Mb
- Less memory
 - < 1 Mb

- Less power consumption
 - < 10 mA draw
 - Can be powered by solar
- Boot time
 - < 1 second

- Other
 - More affordable (couple of \$\$)
 - Code needs to be compiled and flashed

Note:

We will be working with the raspberry PI for these lessons since it is much easier to develop code and quicker to iterate

however...

I will frequently refer to it as a microcontroller, since that is how we will be using them

GPIO Basics

“General Purpose Input/Output”

Raspberry Pi B+, 2, 3, 4, 5 & Zero

3V3	1	2	5V
GPIO2	3	4	5V
GPIO3	5	6	GND
GPIO4	7	8	GPIO14
GND	9	10	GPIO15
GPIO17	11	12	GPIO18
GPIO27	13	14	GND
GPIO22	15	16	GPIO23
3V3	17	18	GPIO24
GPIO10	19	20	GND
GPIO9	21	22	GPIO25
GPIO11	23	24	GPIO8
GND	25	26	GPIO7
DNC	27	28	DNC
GPIO5	29	30	GND
GPIO6	31	32	GPIO12
GPIO13	33	34	GND
GPIO19	35	36	GPIO16
GPIO26	37	38	GPIO20
GND	39	40	GPIO21

Key
+
Ground
UART
I2C
SPI
GPIO
Pin Number



- GPIO pins are how the microcontroller (MCU) sends and receives data from devices
- UART, I2C, and SPI are different communication protocols
 - It's all the same data, but it is structured and sent in different ways
- An input pin will read voltage, an output pin will set voltage
- Discrete vs analog signals
 - Discrete = 0 or 1
 - $> 2.3 \text{ V} \rightarrow \text{TRUE}$
 - $< 1 \text{ V} \rightarrow \text{FALSE}$
 - Between 1 and 2.3 V $\rightarrow$ logic bugs
 - Analog = anywhere between 0 V and 3.3 V

Most “analog” signals are not truly analog

Typically, you need a separate DAC (digital-to-analog chip) to create a true analog signal.

Most MCUs use PWM (pulse-width modulation) on an output pin so it mimics an analog output by changing the duty cycle.

The PWM frequency is >100 Hz, and for most applications this is indistinguishable from a steady input.

Example:

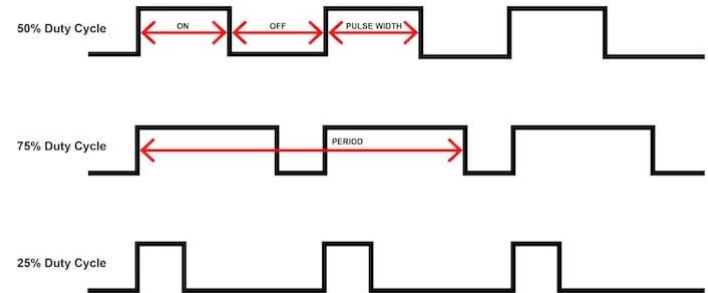
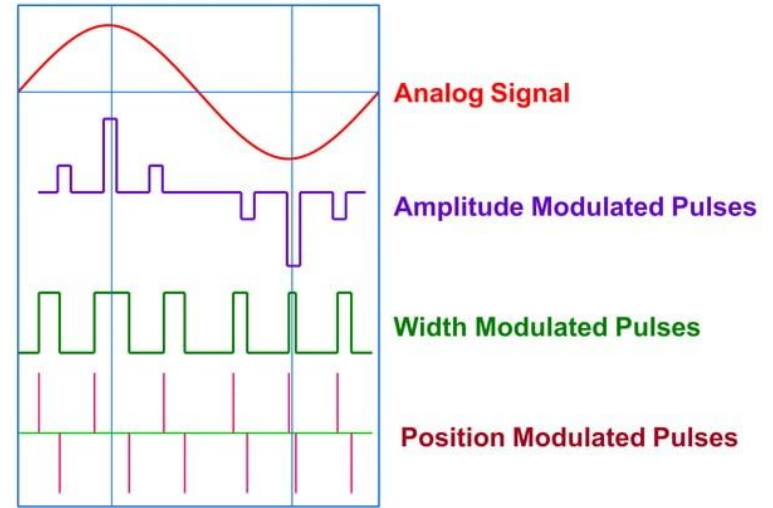
We have an output pin connected to a motor, and the motor's speed is determined by the voltage it receives (0 - 5 V).

If we want to run the motor at 25% speed, it needs to receive 1.25 V across its terminals.

We then set the pin's value to 0.25, causing it to switch between 0 and 5 V on a 25% duty cycle

As far as the motor is concerned, this is the same as 1.25 V

PAM, PWM and PPM at a glance:



Create a controller for a traffic light



Objective 1: Use a green, amber, and red LED, and get them to cycle in the correct order.
The exact timing is up to you.

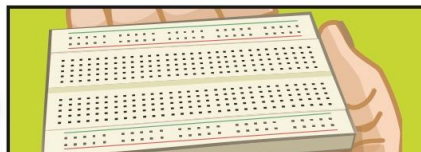
Objective 2: Use a button input to simulate a car pulling up.
This input should trigger a light change.
Be mindful of button “bouncing”

Objective 3: Create another traffic light for cross-traffic.
This trio of LEDs should switch alongside the original light without creating an unsafe condition (i.e. light is green in both directions, enough time for cars to stop, etc...)

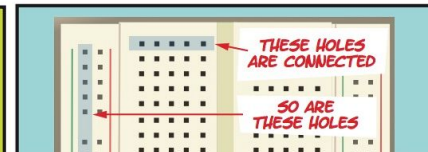
What is a BREADBOARD?

(and how do I use it?)

BY JODY CULKIN & **Make:** makezine.com



A **SOLDERLESS BREADBOARD** IS A PLASTIC BOX FULL OF METAL STRIPS, WITH A GRID OF HOLES ON TOP. IT IS USED TO BUILD AND TEST CIRCUITS QUICKLY.



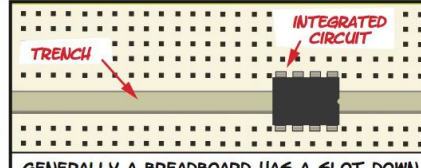
THE HOLES IN THE BREADBOARD ARE CONNECTED IN PATTERNS THAT ALLOW YOU TO ATTACH **COMPONENTS** TO EACH OTHER AND TO **POWER** AND **GROUND**.



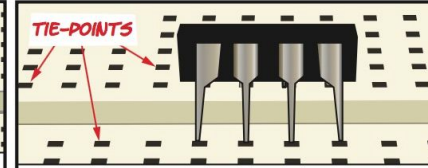
COMPONENTS ARE PARTS USED TO BUILD OUR CIRCUIT.



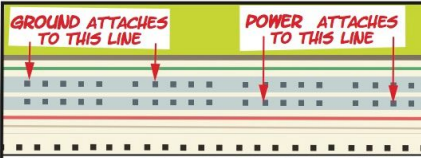
WE WILL ATTACH A **BATTERY** OR OTHER **POWER SOURCE** TO **POWER** AND **GROUND** TO RUN OUR CIRCUIT.



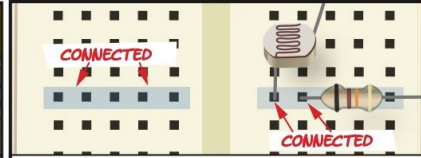
GENERALLY A BREADBOARD HAS A SLOT DOWN THE MIDDLE CALLED A **TRENCH**. THE WIDTH IS DESIGNED SO MANY **INTEGRATED CIRCUITS (ICS)** FIT RIGHT ACROSS IT.



THE HOLES, CALLED **TIE-POINTS**, ARE THE SAME DISTANCE APART AS THE PINS ON MANY **ICS** AND OTHER COMPONENTS.



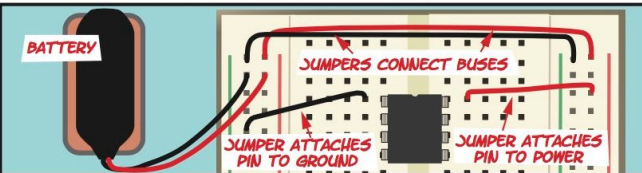
THERE ARE OFTEN 2 LINES OF HOLES ON EACH OF THE LONG SIDES OF THE BOARD THAT GET ATTACHED TO **POWER** AND **GROUND**. THESE LINES ARE CALLED **DISTRIBUTION BUSES**.



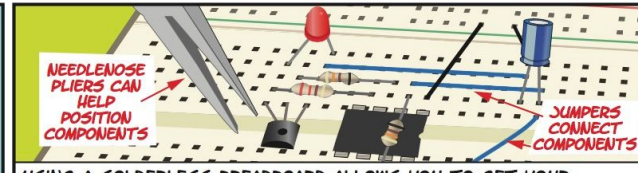
ROWS OF FIVE CONNECTED TIE-POINTS RUN PERPENDICULAR TO THE **BUSES**. TO CONNECT COMPONENTS, PUT THEIR LEADS IN TIE-POINTS IN THE SAME ROW.



HERE A **RESISTOR** IS CONNECTED TO **POWER** AND TO THE **ANODE** OF AN **LED**. THE OTHER LEAD OF THE **LED** ATTACHES TO **GROUND**.



YOU CAN MAKE CONNECTIONS BETWEEN ROWS OF TIE-POINTS BY INSERTING SHORT PIECES OF WIRE CALLED **JUMPERS** BETWEEN HOLES IN DIFFERENT ROWS.



USING A **SOLDERLESS BREADBOARD** ALLOWS YOU TO GET YOUR CIRCUIT UP AND RUNNING QUICKLY SO YOU CAN TEST IT. ONCE YOU HAVE IT JUST RIGHT, YOU CAN BUILD A MORE PERMANENT VERSION ON **PERFBOARD** OR A **PCB**!